

Synthetic Data Generator

Last updated: 2026-06-03

The Synthetic Data Generator is an admin-only tool for populating an Airtable base with realistic-looking test data. It creates Products, Assets, and Work records (with configurable link fields) against a studio-configured "target" base. It is used to build test environments for ArtHound without needing real production data.

Access

Backend — org admin. All routes are under `/api/synthetic` and call `require_admin(user)`, which authorizes any user holding the `owner` or `admin` role in their org (studio **or** vendor). It is *not* gated on platform-admin email.

UI — behind the platform-admin gate. The generator is a "Synthetic data" card on the platform admin page (`AdminPanel.jsx` at `/admin`); its **Open wizard** button launches the `SyntheticDataModal` component. That page is reached only via the **Platform Admin** button in the Topbar, which renders solely for platform admins (`PLATFORM_ADMIN_EMAILS`). So although the backend would authorize any org admin, the only UI entry point currently sits behind the platform-admin gate.

Concepts

Target: a saved connection to an Airtable base with a P→A→W table mapping. Stored in `synthetic_targets` (studio-scoped). Holds an encrypted PAT token; the plaintext is never returned from any endpoint.

Table mapping: for each entity tier (Product, Asset, Work): - Which table to create records in (`p_table`, `a_table`, `w_table`) - Which field is the primary name field (`p_primary_field`, etc.) - Which field is the link field for hierarchy connections (`a_link_field`, `w_link_field`), optional

Extra fields: a list of additional fields (with type and options) to populate with random values on each created record. The generator only writes to writable field types (see below).

Setup Flow

Step 1: Connect a base

`POST /api/synthetic/targets`, provide a name, Base ID, and PAT token. The route validates the credentials by fetching the base schema; if the base is reachable and has tables, the connection is saved.

The token is encrypted at rest using the same `lib/crypto` mechanism as source credentials.

Step 2: Map tables and fields

`GET /api/synthetic/targets/{target_id}/schema`, fetches the live table and field list from Airtable. The admin uses these to configure which tables correspond to Products, Assets, and Work.

`PATCH /api/synthetic/targets/{target_id}`, saves the table/field mapping. This is required before generation can run.

Step 3: Generate records

`POST /api/synthetic/generate` with:

```
{
  "target_id": "uuid",
  "p_count": 3,
  "a_count": 50,
  "w_count": 150,
  "link_a_to_p": true,
  "link_w_to_a": true,
  "extra_fields_p": [],
  "extra_fields_a": [
    { "name": "Complexity", "type": "singleSelect", "options": { "choices": [{"name": "Low"}] }
  ],
  "extra_fields_w": []
}
```

- `link_a_to_p`, if true, each Asset is linked to a random Product via `a_link_field`
- `link_w_to_a`, if true, each Work item is linked to a random Asset via `w_link_field`
- Counts are capped at 1-1000 per entity type

Generation proceeds in order: Products → Assets → Work. Each tier is batched in groups of 10 and written to Airtable with a 200ms pause between batches (staying under Airtable's 5 req/s limit).

Generated Content

Names are built from three word lists: nouns (`Phoenix`, `Ember`, `Cobalt`, ...), verbs (`Render`, `Scatter`, `Forge`, ...), and adverbs (`Swiftly`, `Boldly`, `Crisply`, ...).

- **Products:** {Noun} {NNN} (e.g. "Phoenix 001")
- **Assets:** {Verb} {NNN} (e.g. "Render 042")
- **Work items:** {Adverb} {NNN} (e.g. "Swiftly 007")

Extra field values are generated randomly based on the declared field type:

Field type	Generation
<code>singleLineText</code>	Random word from noun/verb/adverb pool
<code>multilineText</code>	Three-word phrase
<code>number</code> , <code>currency</code> , <code>percent</code>	Random integer 1-100 (or float if precision > 0)
<code>singleSelect</code>	Random choice from declared <code>choices</code>
<code>multipleSelects</code>	1-2 random choices
<code>checkbox</code>	Random true/false
<code>date</code>	Random date within the past year
<code>dateTime</code>	Random datetime within the past year
<code>rating</code>	Random integer 1 to <code>max</code>
<code>duration</code>	Random seconds between 1 min and 2 hrs
<code>email</code>	<code>{verb}. {noun}@example.com</code>
<code>url</code>	<code>https://example.com/{noun}</code>
<code>phoneNumber</code>	US-format random number

Writable types only: field types not in the writable set (`singleLineText`, `multilineText`, `number`, `currency`, `percent`, `singleSelect`, `multipleSelects`, `checkbox`, `date`, `dateTime`, `rating`, `duration`, `email`, `url`, `phoneNumber`) are silently dropped before the batch write. Formula fields, rollup fields, computed fields, and attachment fields are never written to.

Error Handling

- **429 (rate limited):** respects `Retry-After` header (default 30s); retries up to 3 times
- **5xx:** exponential backoff (2 → 4 → 8s); retries up to 3 times
- **401 / 403:** immediate failure with a descriptive message
- **422:** Airtable rejected the payload (e.g. invalid link field name); immediate failure

If generation fails part-way through, the response is `HTTP 207` with a partial `{products: N, assets: N, work: N, error: " ... "}` body. Records already written to Airtable are not rolled back.

API Reference

All endpoints are under `/api/synthetic`. Admin-only.

Method	Path	Description
GET	/targets	List saved targets for the studio
POST	/targets	Save a new target (validates credentials)
GET	/targets/{id}/schema	Fetch live table + field list from Airtable
PATCH	/targets/{id}	Update table/field mapping
DELETE	/targets/{id}	Delete a target
POST	/generate	Create records in Airtable

Database Schema

synthetic_targets

```

id          uuid PK
studio_id   uuid → studios
name        text
base_id     text      (Airtable Base ID)
token_enc   text      (encrypted PAT)
p_table     text      (Airtable table name for Products)
a_table     text      (Airtable table name for Assets)
w_table     text      (Airtable table name for Work)
p_primary_field text
a_primary_field text
w_primary_field text
a_link_field text      (optional – link from Asset to Product)
w_link_field text      (optional – link from Work to Asset)
created_at  text

```

Known Gaps

No sync trigger after generation: records are written to Airtable but ArtHound does not automatically trigger a sync to ingest them. A manual sync run (or waiting for the poll loop) is required to see the generated records in the Asset Viewer.

No field preset support: extra fields must be specified per-generation-run. There is no way to save a reusable field configuration for a target.

Link field names are table-name strings, not IDs: `a_link_field` and `w_link_field` are the display names of Airtable linked-record fields. If a field is renamed in Airtable, the saved target mapping silently stops linking.